

Programiranje za web

Predavanje 5: Oblikovanje API-ja i CRUD domene

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

05.11.2026.



Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

- 1 Od zapisa do resursa
- 2 Sheme
- 3 Popisi
- 4 Od modela do tablice

Dosad je aplikacija znala tko je korisnik i što smije. Sada dobiva zapise nad kojima radi i sučelje kojim ih netko drugi može stvarati, čitati, mijenjati i brisati.

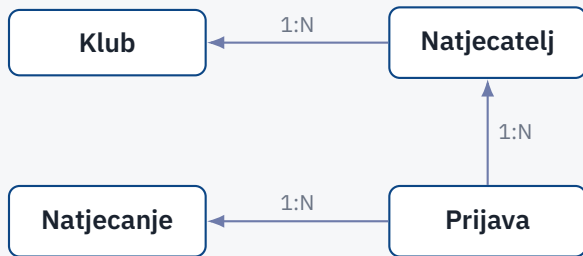
Ishodi učenja

Nakon ovog predavanja student može:

- preslikati četiri osnovne operacije nad zapisom u metode i statusne kodove
- obrazložiti izbor između ugniježdene i samostalne rute
- napisati odvojene sheme za unos, izmjenu i odgovor i reći zašto se razlikuju
- dodati provjeru koja nadilazi tip polja i reći gdje je granica sheme
- ograničiti veličinu odgovora straničenjem i filtriranjem kroz parametre upita
- objasniti zašto se povezani zapisi moraju zatražiti unaprijed
- razlikovati `flush` od `commit` i reći gdje transakcija počinje i završava

Domena primjera

Sustav prijava klubova na powerlifting natjecanje. Uz klub i račun dolaze tri nova zapisa:



Strelica pokazuje na nadređenu stranu: natjecatelj pripada klubu, prijava jednom natjecatelju i jednom natjecanju.

Prijava nije samo spojnica dvaju zapisa. Nosi i vlastite podatke, težinsku kategoriju i najbolji zbroj (`total`), pa je i sama zapis o kojem se vodi računa.

Od zapisa do resursa

Što je resurs, četiri operacije nad njim, pripadnost u adresi, prava koja ruta objavljuje.

Zapis i resurs

Zapis u bazi nema adresu. Postoji kao redak i do njega se dolazi upitom koji zna kako je tablica složena.

Sučelje prema klijentu ne smije tražiti to znanje. Zato se svaka vrsta zapisa objavljuje kao **resurs**: nešto što ima svoju adresu i skup radnji koje se nad njom izvode.

U bazi	Prema klijentu
tablica <code>lifters</code>	resurs na <code>/lifters</code>
jedan redak	<code>/lifters/31</code>
stupac <code>first_name</code>	polje u tijelu zahtjeva i odgovora

Preslikavanje ne mora biti jedan na jedan. Resurs smije spojiti podatke iz više tablica, izostaviti stupce koje nitko izvana ne treba, ili uopće ne odgovarati nijednoj tablici.

Četiri operacije nad zapisom

Nad svakim se resursom radi isto: stvara se, čita, mijenja i briše. Te četiri operacije nose zajednički naziv CRUD (engl. *create, read, update, delete*) i preslikavaju se u metode i odgovore:

Operacija	Metoda	Kod	Tijelo odgovora
popis	GET	200	niz zapisa
jedan zapis	GET	200	zapis
stvaranje	POST	201	stvoreni zapis
izmjena	PATCH	200	izmijenjeni zapis
brisanje	DELETE	204	nema ga

Stvaranje vraća zapis jer klijent prije odgovora ne zna oznaku koju je baza dodijelila. Brisanje ne vraća ništa, pa uz 204 tijela nema.

Konvencija

Ista se tablica primjenjuje na svaki resurs, pa tko zna kako radi jedan, zna i ostale. To je odluka pri oblikovanju sučelja, ne zahtjev protokola.

Pripadnost u adresi

Natjecatelj uvijek pripada nekom klubu. Ta se pripadnost u adresi može izraziti na dva načina:

Oblik	Adresa
ugniježđena	<code>/clubs/12/lifters</code>
samostalna	<code>/lifters?club_id=12</code>

U prvom obliku pripadnost je dio adrese, pa je uvijek prisutna i ne može se izostaviti. U drugom je uvjet pretrage: izostavi li se, ruta vraća natjecatelje svih klubova, što je upravo ono što provjera vlasništva mora spriječiti.

Izbor, ne pravilo

Ugniježđeni oblik odgovara odnosu u kojem podređeni zapis bez nadređenoga nema smisla. Samostalni je prikladniji kad se zapis dohvaća po više različitih kriterija, jer bi inače zahtijevao više ugniježđenih adresa za istu vrstu zapisa. Ovdje su natjecatelji ugniježđeni pod klubom, a natjecanja stoje samostalno.

Gdje se ugnježđivanje zapisuje

Ruta ne nosi cijelu adresu. Datoteka s rutama natjecatelja zna samo za / i /{lifter_id}, a ostatak dolazi pri uključivanju u aplikaciju:

```
# app/main.py
app.include_router(clubs_router, prefix="/clubs")
app.include_router(lifters_router,
                    prefix="/clubs/{club_id}/lifters")
app.include_router(competitions_router, prefix="/competitions")
app.include_router(registrations_router,
                    prefix="/competitions/{comp_id}/registrations")
```

Time je oblik sučelja vidljiv na jednom mjestu. Iz ovog se popisa čita koja je pripadnost ugnježđena, bez otvaranja ijedne datoteke s rutama.

Oznaka iz prefiksa dolazi u funkciju rute kao običan argument, pa club_id ne treba nigdje posebno izvlačiti.

Pravo koje ruta objavljuje

Natjecanja čita svatko tko je prijavljen, a mijenja ih samo administrator. Ta se razlika ne vidi u tijelu funkcije nego u njezinu potpisu:

```
# app/routers/competitions.py
@router.get("/", response_model=list[CompetitionResponse])
async def list_competitions(
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    return await competition_service.list_competitions(db)

@router.post("/", response_model=CompetitionResponse, status_code=201)
async def create_competition(
    body: CompetitionCreate,
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_role("admin")),
):
    return await competition_service.create_competition(db, body)
```

Čitanje traži samo utvrđen identitet, stvaranje traži ulogu. Pravo je time dio opisa rute, pa se čita zajedno s metodom i putanjom, a ne traži se po kodu.

Scheme

Zašto jedna shema nije dovoljna, unos i odgovor, izmjena i neposlana polja, provjere koje nadilaze tip, dokle shema seže.

Isti zapis, različit skup polja

Natjecatelj je u bazi jedan zapis, ali ono što putuje u tijelu zahtjeva i u tijelu odgovora nije isti skup polja. Što stoji u adresi, u tijelo ne ulazi:

Polje	Stvaranje	Izmjena	Odgovor
id	nema ga	u adresi	da
ime i prezime	obavezno	ako se mijenja	da
datum rođenja	obavezno	ako se mijenja	da
spol	obavezno	ako se mijenja	da
club_id	u adresi	u adresi	da

Jedna shema za sve troje ne prolazi: morala bi ista polja istodobno tražiti i ne tražiti, a uz to sadržavati id koji klijent pri stvaranju nema čime ispuniti.

Zato natjecatelj ima tri sheme: `LifterCreate`, `LifterUpdate` i `LifterResponse`.

Konvencija

Broj shema nije propisan nego slijedi iz broja različitih situacija. Prijava u ovom projektu ima i četvrtu, smanjeni oblik natjecatelja unutar odgovora, a prijava korisnika nema nijednu od te tri.

Shema za unos i odgovor

```
# app/schemas/lifter.py
class LifterCreate(BaseModel):
    first_name: str = Field(min_length=1, max_length=80)
    last_name: str = Field(min_length=1, max_length=80)
    birth_date: date
    gender: Literal["M", "F"]

class LifterResponse(BaseModel):
    id: int
    first_name: str
    last_name: str
    birth_date: date
    gender: str
    club_id: int

model_config = {"from_attributes": True}
```

Ulazna shema nema `id` ni `club_id`: prvi dodjeljuje baza, a drugi je već u adresi rute. Postavka `from_attributes` dopušta da se izlazna shema popuni izravno iz zapisa koji je vratio ORM.

Izlazna shema pritom nije samo opis. Odgovor prolazi kroz nju, pa polje koje u njoj ne stoji ne može izaći ni greškom.

Izmjena i neposlana polja

```
# app/schemas/lifter.py
class LifterUpdate(BaseModel):
    first_name: str | None = Field(default=None, min_length=1, max_length=80)
    last_name: str | None = Field(default=None, min_length=1, max_length=80)
    birth_date: date | None = None
    gender: str | None = Field(default=None, pattern=r"^[MF]$")

# app/services/lifter_service.py
for field, value in body.model_dump(exclude_unset=True).items():
    setattr(lifter, field, value)
```

Sva su polja neobavezna, pa klijent šalje ona koja mijenja. Time nastaje dvoznačnost: izostalo polje i polje poslano kao null u shemi izgledaju jednako, a znače različito. `exclude_unset` razlikuje to dvoje: u rječnik ulaze samo polja koja su doista poslana. Bez toga bi izmjena jednog polja obrisala sva ostala. Prazno tijelo shema propušta, pa zapis ostaje nepromijenjen, a odgovor je i dalje 200.

PATCH i PUT

Metoda	Klijent šalje	Ishod
PUT	cijeli zapis	zapis je zamijenjen poslanim
PATCH	samo izmijenjeno	ostala polja ostaju kakva su bila

Razlika je posljedica, ne ukus: PUT je idempotentan jer isti zahtjev ponovno daje isto stanje, a PATCH nije nužno, jer ishod ovisi o tome kakav je zapis zatekao.

PUT pritom traži da klijent poznaje cijeli zapis. Pošalje li nepotpun, izostavljena polja se gube. Ovdje se zato svugdje koristi PATCH.

Provjere koje nadilaze tip

Tip polja hvata pogrešan oblik, ali ne i besmislenу vrijednost. Datum rođenja u budućnosti je ispravan datum:

```
# app/schemas/lifter.py
class LifterCreate(BaseModel):
    birth_date: date

    @field_validator("birth_date")
    @classmethod
    def must_be_past(cls, v: date) -> date:
        if v >= date.today():
            raise ValueError("Datum rođenja mora biti u prošlosti")
        return v
```

Isto vrijedi za kategoriju prijave. Ona je niz znakova, jer postoje i oznake poput 120+, pa tip ne može nabrojiti dopuštene vrijednosti. Validator provjerava je li poslana vrijednost u popisu i imenuje je u poruci ako nije.

Kad takva provjera padne, odgovor ima isti oblik kao i pogreška u tipu: imenuje polje i navodi razlog. Sloj s poslovnim pravilima time nikada ne prima vrijednost koja nema smisla.

Dokle shema seže

Shema vidi poslani sadržaj i ono što proces sam zna, primjerice današnji datum. Može usporediti i dva polja istog zahtjeva:

```
# app/schemas/competition.py
@field_validator("final_deadline")
@classmethod
def final_after_prelim(cls, v, info):
    prelim = info.data.get("prelim_deadline")
    if prelim and v <= prelim:
        raise ValueError("final_deadline mora biti nakon prelim_deadline")
    return v
```

Ono što ne može jest pogledati u bazu. Validator je obična funkcija bez sesije, a u asinkronom radu unutar nje nema ni čekanja odgovora baze.

Granica je time jasna: shema prosuđuje vrijednost samu za sebe, ne u odnosu na druge zapise. Je li natjecatelj već prijavljen, bez baze se ne zna, i to je posao sloja iznad nje.

Odgovor koji nosi povezani zapis

Popis prijava u kojem stoji samo `lifter_id` klijentu ne znači mnogo; za svako ime trebao bi novi zahtjev. Zato odgovor nosi i sam povezani zapis, ali suženo:

```
# app/schemas/registration.py
class LifterBrief(BaseModel):
    first_name: str
    last_name: str

    model_config = {"from_attributes": True}

class RegistrationResponse(BaseModel):
    id: int
    lifter_id: int
    competition_id: int
    category: str
    total: int
    status: str
    lifter: LifterBrief
```

LifterBrief je isti natjecatelj s manje polja. Datum rođenja i klub ovdje nikome ne trebaju, a shema odgovora jedino je mjesto na kojem se taj izbor provodi.

Popisi

Zašto popis ne vraća sve, straničenje i filtriranje, redoslijed, povezani zapisi u popisu.

Popis ima granicu

Ruta koja vraća popis prvo mora odgovoriti koliko zapisa vraća. Odgovor „sve” radi dok ih je malo, a s rastom kvari tri stvari odjednom: upit čita cijelu tablicu, odgovor raste, a klijent prima više nego što može prikazati. Zato granica ulazi odmah:

```
# app/routers/lifters.py
@router.get("/", response_model=list[LifterResponse])
async def list_lifters(
    club_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
    gender: str | None = Query(default=None, pattern=r"^[MF]$"),
    limit: int = Query(default=20, le=100),
    offset: int = Query(default=0, ge=0),
):
    ...
```

limit kaže koliko zapisa, offset koliko ih preskočiti. Vraćanje popisa u dijelovima naziva se straničenje (engl. *pagination*).

Zadane vrijednosti i gornja granica

- **Obje vrijednosti imaju zadanu.** Klijent koji o njima ne zna ništa ipak dobiva ograničen odgovor, pa se ruta ne može slučajno koristiti neograničeno.
- **Gornja granica 1e=100 je zaštita.** Bez nje klijent zatraži milijun zapisa i ruta se vraća na ponašanje s početka.
- **Filtar prolazi istu provjeru kao tijelo.** `gender` ima uzorak `^[MF]$,` pa neispravna vrijednost ne dolazi do upita.

Parametri upita time nisu iznimka: opisani su jednako strogo kao i polja sheme, i jednako ih odbija framework prije nego funkcija počne raditi.

Konvencija

Nazivi `limit` i `offset` nisu propisani. Sučelja koriste i `page` i `per_page`, a neka vraćaju i ukupan broj zapisa uz popis. Bitno je da izbor bude isti na svim rutama.

Upit koji to izvodi

```
# app/repositories/lifter_repo.py
stmt = select(Lifter).where(Lifter.club_id == club_id)
if gender is not None:
    stmt = stmt.where(Lifter.gender == gender)
stmt = stmt.order_by(Lifter.id).limit(limit).offset(offset)
```

Upit se gradi postupno: uvjet pripadnosti stoji uvijek, uvjet spola samo ako je filter poslan. Sloj repositories time ostaje jedino mjesto koje zna kako se do zapisa dolazi.

order_by nije ukras. Bez zadanog redoslijeda baza ga ne jamči, pa isti zapis može doći i na prvoj i na drugoj stranici, dok drugi ne dođe ni na jednu.

Konvencija

Uz offset baza pri svakom zahtjevu preskače sve prethodne retke, pa duboke stranice postaju spore. Drugi pristup pamti oznaku zadnjeg viđenog zapisa i traži one iza nje. Brži je i otporan na umetanje, ali ne dopušta skok na proizvoljnu stranicu. Ovdje se koristi offset, jer su popisi kratki.

Povezani zapis u popisu

Prijave se vraćaju zajedno s natjecateljem. Veza opisana u modelu dopušta da se povezani zapis čita kao svojstvo, primjerice `registration.lifter`, a ORM ga tada dohvaća zasebnim upitom.

U asinkronom radu to ne prolazi: čitanje svojstva nije mjesto na kojem se smije čekati odgovor baze, pa umjesto podataka slijedi iznimka. Povezani se zapisi zato traže unaprijed, u istom upitu:

```
# app/repositories/registration_repo.py
stmt = (
    select(Registration)
    .options(selectinload(Registration.lifter))
    .where(Registration.competition_id == comp_id)
)
```

`selectinload` znači: dohvati prijave, pa jednim dodatnim upitom sve njihove natjecatelje. Dakle dva upita umjesto jednog po prijavi. Bez toga bi popis od pedeset prijava izveo pedeset i jedan upit, što se u razvoju s dvjema prijavama ne primijeti.

Od modela do tablice

Zapis opisan u kodu, migracija koja tablicu stvara, upis i granica transakcije.

Zapis opisan u kodu

Svaki resurs ima svoj model. Najviše govori prijava, jer ona nosi i vezu i vlastite podatke:

```
# app/models/registration.py
class Registration(Base):
    __tablename__ = "registrations"
    __table_args__ = (
        UniqueConstraint("lifter_id", "competition_id",
                        name="uq_lifter_competition"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    lifter_id: Mapped[int] = mapped_column(ForeignKey("lifters.id"))
    competition_id: Mapped[int] = mapped_column(
        ForeignKey("competitions.id"))
    )
    category: Mapped[str] = mapped_column(String(20), nullable=False)
    total: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    registered_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now())
    )
```

Dva strana ključa nose vezu, category i total vlastite podatke, a registered_at popunjava baza, pa vrijeme upisa ne ovisi o satu poslužitelja. Ograničenje jedinstvenosti kaže da isti natjecatelj ne može biti dvaput na istom natjecanju.

Migracija koja tablicu stvara

Model opisan u Pythonu još nije tablica. Alembic uspoređuje stanje baze s onim što vidi u `Base.metadata`, a tamo dolaze samo razredi koji su negdje uvezeni:

```
# app/models/__init__.py
from app.core.database import Base
from app.models.club import Club
from app.models.competition import Competition
from app.models.lifter import Lifter
from app.models.registration import Registration
from app.models.user import User
```

Izostane li redak, `alembic revision --autogenerate` neće prijaviti pogrešku. Napraviti će praznu migraciju, jer razliku između modela i baze nije ni vidio. Nova tablica tako ne nastaje pokretanjem aplikacije nego migracijom, jednako kao i prva tablica u projektu.

Upis i granica transakcije

```
# app/repositories/lifter_repo.py
async def create(db: AsyncSession, lifter: Lifter) -> Lifter:
    db.add(lifter)
    await db.flush()
    return lifter

# app/core/deps.py
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

`db.add` najavljuje zapis sesiji, `db.flush` šalje naredbu upisa bazi, pa zapis dobiva oznaku i vidljiv je daljnjim upitima. `commit` ga čini trajnim i izvodi se tek kad cijela obrada prođe bez iznimke.

Odatle pravilo koje vrijedi kroz cijeli projekt: **jedan zahtjev je jedna transakcija**. Da repozitorij sam radi `commit`, niz od dvaju upisa ne bi bio jedna cjelina, pa bi nakon pogreške dio zapisa ostao upisan.

Sažetak

- Resurs je vrsta zapisa objavljena pod vlastitom adresom; preslikavanje iz tablice ne mora biti jedan na jedan.
- Nad svakim se resursom ponavljaju iste četiri operacije; stvaranje vraća zapis jer oznaku dodjeljuje baza, brisanje ne vraća ništa.
- Ugniježđena adresa nosi pripadnost i ne može se izostaviti; kod samostalne je pripadnost uvjet pretrage i mora se provjeriti kao i svaki drugi.
- Unos, izmjena i odgovor tri su različite sheme, a `exclude_unset` razlikuje neposlano polje od poslanog praznog.
- Shema prosuđuje vrijednost samu za sebe; pravilo koje traži uvid u bazu pripada sloju iznad nje.
- Popis se vraća u dijelovima, s gornjom granicom i zadanim redoslijedom; povezani zapisi traže se unaprijed.
- `flush` šalje upis, `commit` ga čini trajnim, a izvodi ga ovisnost koja drži sesiju. Jedan zahtjev je jedna transakcija.

Što slijedi

- **12.11.2026.** Predavanje 6, poslovna pravila: faze natjecanja, rokovi i operacije koje o njima ovise.
- **19.11.2026.** Obrana prvoga dijela projekta.
- **26.11.2026.** Predavanje 7, arhitektura jednostranične aplikacije.

Do sljedećega predavanja: dodati natjecatelja svojem klubu, prijaviti ga na natjecanje, pa pozvati popis prijava s `limit=1` i usporediti odgovor s onim bez parametra.